Episode 1: Mistakes, Errors, Scenarios and Interview Answers

Common wrong ideas, errors beginners see, real-life scenarios, and solid interview answers with diagrams

Part A: Common misunderstandings

Almost every beginner believes at least one of these. Check yourself!

Wrong idea	Why it is wrong	Correct idea
"A container is just a small virtual machine."	A VM carries its own full operating system. A container does not.	Containers share the host's OS, which makes them light and fast.
"Image and container are the same thing."	They are related but different.	Image = read-only blueprint. Container = a running copy made from it.
"Docker makes my app run faster."	Docker does not speed up your code. It makes the environment consistent.	Docker's main gift is **consistency** and quick start-up, not faster code.
"If it works in Docker, my app has no bugs."	Docker packs your app as it is, bugs included.	Docker removes environment surprises, not coding mistakes.
"Docker and Docker Hub are the same."	Docker is the tool. Docker Hub is one online registry.	Docker runs containers; a registry stores images. Others exist too.
"Docker replaces every virtual machine."	Some jobs need a full separate OS or very strong isolation.	Containers and VMs often work together. Choose by need.
"Docker is a programming language."	You do not write programs in Docker.	Docker is a tool that packages and runs apps written in any language.
"Only developers need to know Docker."	Ops, QA and DevOps people deploy and test containers daily.	Docker is a shared language between development and operations.

Part B: Errors beginners see with docker run hello-world

Episode 1's demo is optional, but you may try it. Here are messages you might meet and what to do. Some are not even errors!

What you see	What it means	What to do
`Unable to find image 'hello-world:latest' locally`	**Not an error.** Docker is telling you it will download the image first.	Do nothing. Wait for the download to finish.
`docker: command not found`	Docker is not installed, or your terminal cannot find it.	Install Docker, or open a new terminal after installing.
`Cannot connect to the Docker daemon at unix:///var/run/docker.sock. Is the docker daemon running?`	The Docker background service is not running.	Start Docker Desktop, or on Linux start the Docker service.

What you see	What it means	What to do
`permission denied while trying to connect to the Docker daemon socket`	Your user is not allowed to talk to Docker.	On Linux, add your user to the docker group (then log out and in) or use sudo.
`pull access denied ... repository does not exist or may require 'docker login'`	The image name is wrong (a typo such as helo-world) or it is private.	Check the spelling. Search the name on Docker Hub.
`net/http: request canceled ... Client.Timeout exceeded`	Docker cannot reach the registry. Often internet, VPN or proxy trouble.	Check your connection, turn off the VPN, try again.
`toomanyrequests: You have reached your pull rate limit`	Docker Hub limits downloads for anonymous users.	Log in with a free account (docker login) or wait and retry.

Good habit: read the **last line** of an error first, then the first line. Copy it exactly into a search engine. Almost every Docker error has been solved by someone before.

Part C: Scenario-based questions

Read each situation and think about your answer before checking Part D. There are no tricks, only real-life thinking.

Scenario 1: Works on my laptop

Neha's Python 3.11 app runs perfectly on her laptop. On the production server (Python 3.8) it crashes. **How would Docker help, and what exactly would she ship?**

Your answer: _____

Scenario 2: The new teammate

A new teammate needs two days to set up the project: installing tools, matching versions, fixing errors. **How could Docker shorten this?**

Your answer: _____

Scenario 3: Fifty small apps

A company runs 50 small apps. With VMs it needs 50 operating systems. A manager asks for a cheaper way. **What do you suggest, and why is it cheaper?**

Your answer: _____

Scenario 4: Three containers, one image

You start three containers from the same image. Inside container 1 you change a file. **Do containers 2 and 3 change? Does the image change?**

Your answer: _____

Scenario 5: The crashing neighbour

Five containers run on one server. One app crashes badly. **What happens to the other four, and which Docker benefit is this?**

Your answer: _____

Scenario 6: Finding a ready-made tool

You need a web server (nginx) quickly and do not want to install it by hand. **Where would you look, and what is that place called?**

Your answer: _____

Scenario 7: Recipe vs lunchbox

Your manager says: "We already send developers a document with setup steps. Why do we need Docker?" **Explain the difference using the recipe and lunchbox idea.**

Your answer: _____

Scenario 8: Image or container?

A colleague says: "I downloaded the container from Docker Hub." **Is that wording correct? What should they say?**

Your answer: _____

Scenario 9: Choosing the right tool

A project must run a full Windows-only program on a Linux server, completely walled off. **Is a container the best choice? What else could you consider?**

Your answer: _____

Scenario 10: The slow start

A test environment as a VM takes 5 minutes to boot every time. The team wants fast, repeatable test setups. **What would you try?**

Your answer: _____

Quick true or false

1. Docker packs an app together with what it needs to run. True / False
2. A container and an image are exactly the same thing. True / False
3. One image can start many containers. True / False
4. Containers usually start slower than virtual machines. True / False
5. Docker Hub is an example of a registry. True / False
6. Docker guarantees your code has no bugs. True / False

Part D: Answer key

Compare your answers. If yours differ a little, that is fine. Look for the main idea.

- **Scenario 1:** She ships a **Docker image** containing the app, Python 3.11 and its libraries. The server only needs Docker, so the server's Python 3.8 no longer matters.
- **Scenario 2:** Give them the image (or a file that builds it) and one command to run it. Versions and tools come inside the package, so setup takes minutes, not days.
- **Scenario 3:** Use **containers**. They share one operating system instead of carrying 50, so they use less memory and disk, start in seconds, and let more apps fit on each machine.
- **Scenario 4: No and no.** Containers are separate from each other. A change inside one container stays there. The image is read-only and never changes.
- **Scenario 5:** The other four keep running. This is **isolation**: one broken app does not take down the rest.
- **Scenario 6:** Look in a **registry**, such as Docker Hub, where ready-made images are stored and shared.
- **Scenario 7:** A document is only instructions (the recipe), so every kitchen can still give a different result. Docker ships the whole sealed lunchbox (app, libraries, settings), so the result is the same everywhere.
- **Scenario 8:** Not quite. They downloaded an **image**. A container is what you get when you **run** an image.
- **Scenario 9:** Probably **not**. Containers share the host's operating system, so a Windows-only program does not fit on a Linux host. A **virtual machine** with its own Windows OS is the better fit.
- **Scenario 10:** Try **containers**. They start in seconds, and because the image is the same every time, the test setup is repeatable.

True or false answers

1. **True** | 2. **False** (image = blueprint, container = running copy) | 3. **True** | 4. **False** (containers start in seconds) | 5. **True** | 6. **False** (Docker removes environment problems, not coding bugs)

How did you score?

Score	What it means
5 to 6 right (true/false) and most scenarios	Great! You are ready for Episode 2.
3 to 4 right	Good start. Re-read the Theory Notes and the Diagram Guide.
Below 3	No problem. Read the Story PDF first, then try again.

Part E: Interview Corner - solid answers with diagrams

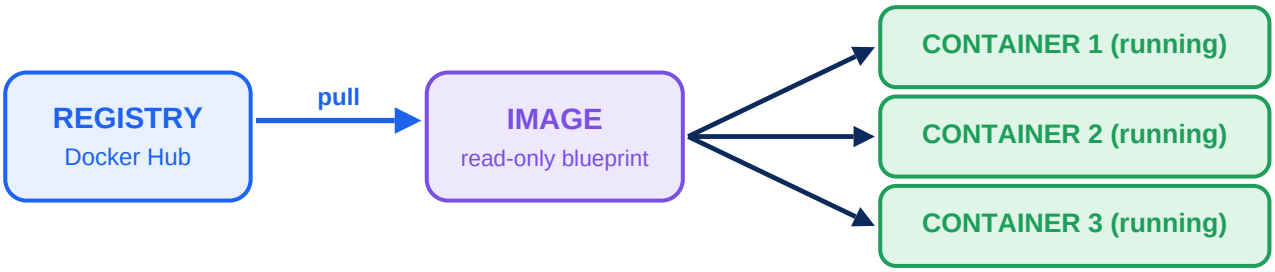
Interviewers ask Docker basics to check whether you truly understand, not whether you memorised words. Use the model answers below as a base and put them in your own words.

How to structure any answer (4 steps): 1. Define it in one clear sentence. **2. Compare or use an analogy** (recipe/dish, houses/apartments). **3. Give a real example** from your work or lab. **4. Mention a trade-off** or limitation. Mentioning a trade-off is what makes an answer sound experienced.

Red flag (avoid saying)	Say this instead
"A container is a small VM."	A container is an isolated process that shares the host kernel.
"Docker makes the code faster."	Docker makes the environment consistent and deployment fast.
"Image and container are the same."	Image is the template; container is a running instance.
"Containers are fully secure."	Isolation is good but weaker than a VM, so apply hardening.

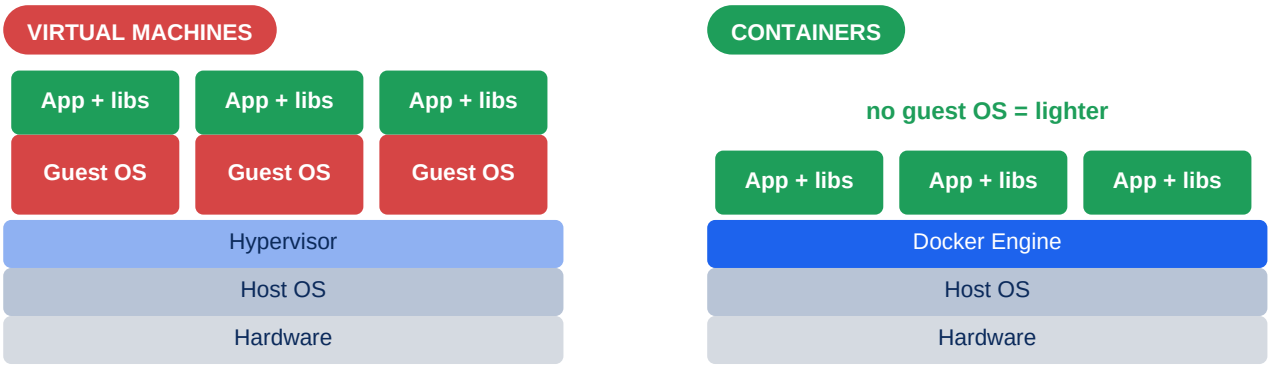
Interview Q1. What is Docker and why do we use it?	
Say this first (30 sec)	Docker is a platform that packages an application with its dependencies (runtime, libraries, settings) into an **image** and runs it as an isolated **container**. Since the image is identical everywhere, the app behaves the same on a laptop, in test and in production. It removes the "works on my machine" problem and makes deployments fast and repeatable.
Add depth	Containers use features of the host Linux kernel, mainly **namespaces** (isolation of processes, network, filesystem) and **cgroups** (limits on CPU and memory). That is why they are light. Key benefits: consistency, fast start, high density, isolation, easy CI/CD.
Example to adapt	"A Python 3.11 service worked in dev but production had 3.8. We shipped it as an image, so the server only needed Docker and the mismatch disappeared." (Replace with your own project.)
Likely follow-up	"Does Docker only work on Linux?" Linux containers need a Linux kernel. On Windows and macOS, Docker Desktop runs a small Linux VM behind the scenes.

Interview Q2. What is the difference between an image and a container?



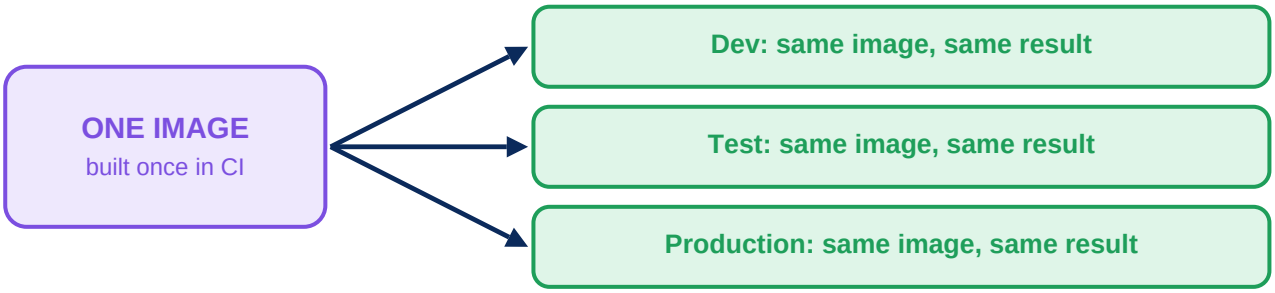
Say this first (30 sec)	An **image** is a read-only template that holds the app and everything it needs. A **container** is a running instance created from that image. Like a class and an object, or a recipe and the dish.
Add depth	One image can start many containers. A container adds a thin writable layer on top of the image. Deleting a container does not delete the image, and nothing done inside a container changes the image.
Example to adapt	"We start 3 containers from one nginx image to handle more traffic. All three are identical."
Likely follow-up	"Can you modify an image?" No. Images are immutable. You build a new image (for example from a Dockerfile) and give it a new tag. We cover this in Episode 3.

Interview Q3. Containers vs virtual machines: what is the difference?

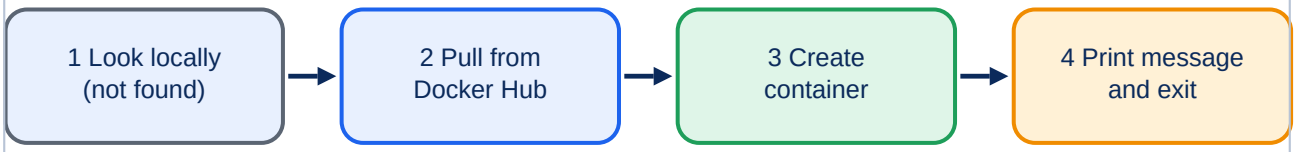


Say this first (30 sec)	A **VM** virtualises hardware. A hypervisor runs several guest operating systems, each with its own kernel. A **container** virtualises at the operating-system level. All containers share the host kernel and are isolated processes. So containers are megabytes and start in seconds; VMs are gigabytes and start in minutes.
Add depth	**Trade-off:** VMs give stronger isolation and can run a different OS kernel. Containers give speed and density but share the kernel. In practice they are combined: containers often run inside VMs in the cloud. Use VMs for untrusted multi-tenant workloads or a different OS; use containers for microservices, CI jobs and app packaging.
Example to adapt	"We replaced 20 small VMs with containers on 3 hosts and reduced cost and start-up time." (Adapt with real numbers only if you have them.)
Likely follow-up	"Is a container a lightweight VM?" No. It is an isolated process, not a full machine with its own kernel.

Interview Q4. What is a registry? Is it the same as Docker Hub?	
Say this first (30 sec)	A **registry** is a service that stores and distributes container images. **Docker Hub** is the best-known public registry, but not the only one.
Add depth	Other registries: GitHub Container Registry (GHCR), Amazon ECR, Google Artifact Registry, Azure Container Registry, or a self-hosted private one. You **pull** images from a registry and **push** your own. An image name looks like `repository:tag`, for example `nginx:1.27`. Use official or verified images and scan them.
Example to adapt	"Our CI builds the image and pushes it to a private registry, and servers pull from there."
Likely follow-up	"Registry vs repository?" The registry is the server. A repository is a collection of versions (tags) of one image, such as nginx.

Interview Q5. SCENARIO: An app works in dev but fails in production. How do you solve and prevent it?	
	
Say this first (30 sec)	I would first confirm it is environment drift (language version, library, config). Then I containerise the app so the runtime and dependencies are inside one image. **Build once, run the same image everywhere.** CI builds and tags the image, and dev, test and production all use it.
Add depth	**Steps:** 1) Pin the base image version, for example python:3.11-slim. 2) Install dependencies inside the image. 3) Keep configuration outside the image (environment variables), so the same image works in each environment. 4) Promote the same tag from test to production. 5) Roll back by redeploying the previous tag. **Limits:** Docker fixes environment drift, not code bugs. Config, secrets and external services can still differ.
Example to adapt	"After containerising, deployment became one command and the environment issues stopped."
Likely follow-up	"Why not bake config into the image?" Then you need a different image for each environment, and secrets leak into image layers. We cover this in Episode 6.

Interview Q6. What exactly happens when you run `docker run hello-world`?



Say this first (30 sec)	Docker checks whether the **hello-world image** is on the machine. If not, it **pulls** it from the registry (Docker Hub). It then **creates a container** from the image and starts it. The container prints a message and exits.
Add depth	This shows the full lifecycle: registry, image, container. Running the same command again skips the download because the image is cached locally, but it creates a **new container** each time. Stopped containers stay on disk until removed.
Example to adapt	"I used it to verify an installation: if hello-world prints its message, the Docker client, daemon and registry access all work."
Likely follow-up	"What if it says the daemon is not running?" Start Docker Desktop or the Docker service. See Part B for other errors.

Interview Q7. Are containers as secure as VMs?

Say this first (30 sec)	Not by default. Containers share the host kernel, so isolation is weaker than a VM. A kernel vulnerability or a risky setup (privileged mode, running as root) can put the host at risk.
Add depth	Hardening: run as a **non-root user**, use small trusted base images, scan images for vulnerabilities, pin versions, never mount the Docker socket into containers needlessly, keep secrets out of images, and update regularly. For high-risk workloads, run containers inside VMs. (Security is Episode 8.)
Example to adapt	"We scanned images in CI and blocked builds with critical vulnerabilities."
Likely follow-up	"What gives containers isolation?" Namespaces and cgroups, plus capabilities and seccomp profiles.

Interview Q8. I deleted a container and my data disappeared. Why, and how do you keep data?

Say this first (30 sec)	Data written inside a container lives in its writable layer, which is **temporary**. When the container is removed, that layer is gone. To keep data, store it in a **volume** or a bind mount that lives outside the container.
Add depth	Typical use: a database container with its data directory on a named volume, so you can replace the container with a new version and keep the data. (Volumes are Episode 4.)
Example to adapt	"The database container was upgraded to a new image version and the data stayed because it was on a volume."
Likely follow-up	"Volume vs bind mount?" A volume is managed by Docker. A bind mount maps a specific host folder.

Interview Q9. Can I run a Linux container on Windows or Mac? What about Windows containers?	
Say this first (30 sec)	Yes, through Docker Desktop, which runs a lightweight **Linux VM** (WSL 2 or Hyper-V on Windows, a VM on Mac) and starts Linux containers inside it. Containers must match the kernel: Linux containers need a Linux kernel, Windows containers need a Windows host.
Add depth	CPU architecture also matters. An image built for amd64 on an Apple Silicon (ARM) laptop may run through emulation and slower, or fail. Multi-architecture images solve this.
Example to adapt	"Our team uses Macs but production is Linux, so we build multi-architecture images."
Likely follow-up	"Then is it really the same everywhere?" Nearly, for the app environment, but kernel and architecture still matter.

Interview Q10. SCENARIO: Two apps on one server need Python 3.8 and Python 3.11. How do you handle it?	
Say this first (30 sec)	Run each app in its **own container** with its own runtime. Each image contains the exact Python version it needs, so they cannot conflict.
Add depth	Without containers you would juggle virtual environments or multiple system installs, which is fragile. With containers, each app also has isolated processes and files, and you map different host ports to each. Resource limits can stop one app starving the other.
Example to adapt	"Legacy service on 3.8 and the new one on 3.11 ran side by side on the same host."
Likely follow-up	"What if both want port 80?" Map them to different host ports or put a reverse proxy in front.

Interview Q11. Why Docker, when we already have setup documents or tools like Ansible?	
Say this first (30 sec)	Documents and manual steps drift and rely on people. Configuration tools like Ansible set up a server, but servers still end up slightly different over time. A Docker image is an **immutable, versioned, tested artifact** that starts in seconds and is identical everywhere.
Add depth	They are complementary: Terraform or Ansible provisions hosts, and Docker packages and runs the applications. Rollback is simple: run the previous image tag.
Example to adapt	"Ansible prepared the hosts and installed Docker, then every release was just a new image tag."
Likely follow-up	"Does Docker replace config management?" No, it replaces app-level installation, not host provisioning.

Interview Q12. Is it okay to use the `latest` tag?	
Say this first (30 sec)	Avoid it in production. `latest` is just the default tag name used when none is given. It is **not** guaranteed to be the newest or the safest, and it can change under you, which breaks reproducibility.
Add depth	Pin an explicit version such as `python:3.11.9-slim` or, for the strongest guarantee, the image **digest**. Using hello-world:latest in a demo is fine.
Example to adapt	"A rebuild pulled a newer base image and broke the build. After pinning versions it was stable."
Likely follow-up	"How do you update pinned versions safely?" Update on purpose, test in CI, then release.

Interview Q13. What are the limitations or downsides of Docker?	
Say this first (30 sec)	Containers share the host kernel (weaker isolation than VMs), persistent data needs extra care, networking and debugging have a learning curve, and running many containers at scale needs an orchestrator such as Kubernetes.
Add depth	Other points: bloated images if built carelessly, security of third-party images, and Linux-vs-Windows kernel limits. Good practice (small images, scanning, volumes, CI) reduces most of these.
Example to adapt	"We moved to multi-stage builds to shrink images and used scanning in CI." (Episodes 7 and 8.)
Likely follow-up	"When would you not use containers?" For strong isolation needs, a different OS kernel, or very simple single-server apps where it adds no value.

Interview Q14. Explain Docker to a non-technical person in 30 seconds.	
Say this first (30 sec)	"Imagine sending a cake recipe to a friend: it often turns out different in their kitchen. Docker is like sending the finished cake in a sealed box, with the oven settings and ingredients included. It tastes the same wherever it is opened. In software, the box is called a container, and it makes sure an app runs the same on every computer."
Add depth	Add one sentence for relevance: "That means fewer failed releases and faster delivery for the team."
Example to adapt	Use the recipe, dish and cookbook-library analogy from the Story PDF.
Likely follow-up	"So what is Docker Hub?" "A library where people share these sealed boxes."

Rapid-fire round (answer in one line)

Question	One-line answer
What is an image?	A read-only template with the app and its dependencies.
What is a container?	A running, isolated instance of an image.
Where are images stored?	In a registry, such as Docker Hub.
Do containers have their own kernel?	No, they share the host kernel.
Which is lighter, VM or container?	Container.
Can one image start many containers?	Yes, as many as you need.
Does deleting a container delete the image?	No.
Is data inside a container permanent?	No, use volumes for persistence.
Does Docker fix bugs in my code?	No, it fixes environment differences.
What problem does Docker mainly solve?	"Works on my machine" environment inconsistency.

Practice tip: Cover the answer column and say each answer out loud, then record yourself on the 3 diagram questions (image vs container, containers vs VMs, hello-world flow). Explaining while drawing the diagram on paper impresses interviewers.